

Day 75: Basic Data Cleaning with Pandas

1. Day Number

Day 75

2. Topic Name

Basic Data Cleaning with Pandas — Missing Values, Duplicates, and Incorrect Data Types

Today you will learn how to fix a few common problems found in real datasets:

- Missing values
- Duplicate rows
- Incorrect data types

You will use simple Pandas tools such as:

```
isna()
dropna()
fillna()
drop_duplicates()
astype()
```

3. Connection to Yesterday

Yesterday, on **Day 74**, you learned how to load and inspect data using Pandas.

You used things like:

```
df.head()
df.shape
df.columns
df.dtypes
```

Inspection helps you discover problems.

Today the workflow becomes:

Load → Inspect → Identify problems → Clean

For example, inspection might reveal:

Product	Price
Pen	20
Book	50
Pencil	NaN
Bag	500
Bag	500

You might notice:

- Pencil has a missing price.
- Bag appears twice.
- Some numeric values might accidentally be stored as text.

That is where **data cleaning** begins.

4. Important Topics

Missing value

A **missing value** means some information is absent.

Example:

Product	Price
Pen	20
Book	NaN

NaN commonly represents a missing value in Pandas.

isna()

isna() helps us find missing values.

Conceptually:

```
df.isna()
```

Pandas produces True where a value is missing and False where it exists.

For example:

Product	Price
False	False
False	True

You can also inspect one column:

```
df["Price"].isna()
```

dropna()

dropna() removes rows containing missing values.

Concept:

```
clean_df = df.dropna()
```

Suppose:

Pen	20
Book	NaN
Pencil	10

After removing rows with missing values:

Pen	20
Pencil	10

This is useful when removing the incomplete row makes sense.

fillna()

Sometimes removing a row would throw away useful information.

Instead, you can replace the missing value.

Concept:

```
df["Price"] = df["Price"].fillna(some_value)
```

For example:

Before:	
Pen	20
Book	NaN
After filling:	
Pen	20
Book	0

But using 0 is only correct if **0 actually makes sense for that data.**

Duplicates

A duplicate occurs when the same row appears more than once.

Example:

Product	Price
Pen	20
Book	50
Book	50

The second Book row may be accidental.

drop_duplicates()

Pandas can remove duplicate rows:

```
df.drop_duplicates()
```

Conceptually:

Before:

Pen	20
Book	50
Book	50

After:

Pen	20
Book	50

Data type conversion

Sometimes a value looks numeric but Pandas stores it as text.

For example:

```
"50"
```

instead of:

```
50
```

These are different Python values.

```
"50"    # string  
50      # integer
```

You may need to convert a column to the correct type.

A simple conversion could use:

```
astype()
```

For example, conceptually:

```
df["Price"] = df["Price"].astype(int)
```

Afterward, Pandas can treat the values as numbers rather than text.

5. Foundational Notes

Cleaning does not mean changing everything

Data cleaning means finding data-quality problems and making **appropriate corrections**.

A useful beginner workflow is:

1. Inspect the data
2. Identify a problem
3. Decide what the problem means
4. Apply one small cleaning operation
5. Inspect again

Do not automatically modify data simply because something looks unusual.

Missing does not always mean zero

Suppose a dataset contains:

Student	Exam Score
Asha	85
Ravi	NaN

You should not automatically replace Ravi's missing score with 0.

Why?

A missing score might mean:

The student was absent.
The score has not been entered yet.
The result was unavailable.

A score of 0 means something different.

Duplicate does not always mean wrong

Imagine sales data:

Product	Price
Pen	20
Pen	20

Those might be duplicates.

But they could also represent **two separate pen purchases**.

So before removing duplicates, understand what one row represents.

Correct data types matter

Suppose prices are stored as:

```
["100", "200", "300"]
```

They are strings.

That can cause problems when you want to calculate:

```
Total  
Average  
Minimum  
Maximum
```

Numbers should normally have an appropriate numeric type.

6. Why Cleaning Decisions Depend on the Meaning of the Data

This is one of the most important ideas in data cleaning.

Consider this missing value:

Product	Stock
Laptop	NaN

What should you do?

You might think:

```
Replace NaN with 0
```

But what does NaN actually mean?

It could mean:

- The laptop has zero stock.
- The stock count hasn't been entered.
- The inventory system failed.
- The product hasn't been checked yet.

Those meanings are very different.

Therefore:

You should understand what the data represents before choosing how to clean it.

Similarly, suppose:

Customer	City
Asha	Bengaluru
Asha	Bengaluru

This could be an accidental duplicate.

But if the table represents individual purchases, Asha may simply have made two purchases.

Cleaning is therefore partly a **technical task** and partly a **reasoning task**.

7. Easy Example

Imagine this small dataset:

Product	Price
Pen	20
Book	50
Pencil	NaN
Book	50
Bag	"500"

There are three simple problems.

Problem 1: Missing value

Pencil → NaN

You need to decide whether to:

<code>dropna()</code>

or:

<code>fillna()</code>

Problem 2: Duplicate

These rows are identical:

Book	50
Book	50

If they truly represent accidental duplication, you could use:

```
drop_duplicates()
```

Problem 3: Incorrect data type

The Bag price may have been entered as:

```
"500"
```

which is text rather than a number.

You may need to convert the price column into a numeric type.

The overall idea is:

```
Inspect
  ↓
Handle missing value
  ↓
Remove genuine duplicate
  ↓
Correct data type
  ↓
Inspect again
```

8. Problem Statement

Create or load a tiny Pandas DataFrame containing product information.

Your dataset should contain:

- At least one normal row
- **One missing value**
- **One duplicate row**
- **One numeric value stored using an incorrect data type**

For example, your table might conceptually contain:

Product	Price
Pen	20
Book	50
Pencil	missing
Book	50
Bag	"500"

Your program should:

1. Display the original DataFrame.
2. Check for missing values.
3. Handle the missing value using either `dropna()` or `fillna()`.
4. Remove the duplicate row.
5. Convert the numeric column to an appropriate numeric data type.
6. Display the cleaned DataFrame.
7. Inspect the data types again.

Keep your cleaning rules simple.

9. Concepts Used

For this exercise, you will use:

- Pandas
- DataFrame
- Rows and columns
- Missing values
- NaN
- `isna()`
- `dropna()` or `fillna()`
- Duplicate rows
- `drop_duplicates()`
- Data types
- Type conversion
- `astype()`
- `dtypes`
- Before-and-after inspection

You are combining yesterday's **inspection skills** with today's **cleaning skills**.

10. Thought Process

Before writing code, think through the problem.

Step 1: Look at the original data

Ask:

What columns exist?
What do the rows represent?
What problems can I see?

Step 2: Find the missing value

Use Pandas to identify where data is missing.

Ask:

```
Should this row be removed?  
OR  
Should the missing value be replaced?
```

For this beginner exercise, choose one simple rule.

Step 3: Look for duplicates

Determine whether repeated rows are genuinely accidental.

For this exercise, assume the exact duplicate was accidentally added.

Remove it.

Step 4: Inspect the data types

Check something like:

```
df.dtypes
```

Ask:

```
Is the Price column numeric?
```

If it is stored incorrectly, convert it.

Step 5: Inspect the cleaned data

Print or inspect the DataFrame again.

Compare:

```
Before cleaning  
vs.  
After cleaning
```

A good habit is:

| Never assume a cleaning operation worked. Inspect the result.

11. Beginner-Friendly Pseudocode

```
START

import pandas

create a small DataFrame

include:
    one missing value
    one duplicate row
    one numeric value stored incorrectly

print the original DataFrame

check for missing values

choose a simple missing-value rule:
    either remove the incomplete row
    OR replace the missing value

remove duplicate rows

inspect the column data types

convert the numeric column to the correct type

print the cleaned DataFrame

print the new data types

END
```

Notice that the pseudocode describes the **logic**, not the exact Python answer.

12. Suggested Solving Approach: Pandas Approach

Use Pandas for each stage.

Think of the exercise as four small jobs:

```
DataFrame
↓
Inspect missing values
↓
Handle missing values
↓
Remove duplicates
```

↓
Correct data types
↓
Cleaned DataFrame

Possible Pandas tools:

```
df.isna()
```

then either:

```
df.dropna()
```

or:

```
df["column"].fillna(...)
```

then:

```
df.drop_duplicates()
```

and finally investigate:

```
df.dtypes
```

before applying an appropriate type conversion.

Do each operation separately while learning rather than trying to clean everything in one long statement.

13. Easy Edge Cases

Edge Case 1: All values are missing

Suppose a column looks like:

```
Price  
NaN  
NaN  
NaN
```

Blindly removing every row could leave you with:

```
Empty DataFrame
```

That may be technically valid, but probably isn't very useful.

You would need to investigate why the entire column is missing.

For now, just remember:

| If almost everything is missing, simple row deletion may not be the best strategy.

Edge Case 2: No duplicates

Your dataset might contain:

Pen	20
Book	50
Pencil	10

Calling:

```
drop_duplicates()
```

would simply leave the rows unchanged.

That is okay.

Cleaning functions do not require a problem to exist.

Edge Case 3: Missing value has an important meaning

Suppose:

Product	Discount
Pen	NaN

NaN might mean:

```
No discount information available
```

That is different from:

```
0% discount
```

So do not automatically convert missing values into zero.

Edge Case 4: Similar rows are not necessarily duplicates

These are not exact duplicates:

Pen	20
Pen	25

The same product appears twice, but the prices differ.

Before deleting one row, you would need more information.

For today's lesson, only deal with an obvious exact duplicate.

14. Expected Before-and-After Result

Your exact values can differ, but the idea should look something like this.

Before cleaning

	Product	Price
0	Pen	20
1	Book	50
2	Pencil	NaN
3	Book	50
4	Bag	500

Conceptually, the dataset contains:

1 missing value
1 duplicate row
1 data-type problem

Your initial data types might show that Price is not stored in the desired numeric type.

After cleaning

Depending on the missing-value rule you choose, you might eventually have something conceptually similar to:

	Product	Price
0	Pen	20
1	Book	50
2	Pencil	...
4	Bag	500

The important result is not the exact row numbers.

You should be able to verify that:

Missing value → handled
Duplicate → removed
Price → stored as an appropriate numeric type

Your program should clearly show a **before** and **after** version so you can see what changed.

15. Hint Only

Start by creating your DataFrame and inspecting these two things:

```
print(df.isna())  
print(df.dtypes)
```

Then solve **one problem at a time**.

Think about this sequence:

```
missing values  
  ↓  
duplicates  
  ↓  
data type
```

Useful methods to investigate:

```
fillna(...)  
dropna()  
drop_duplicates()  
astype(...)
```

For the missing value, choose either **fill it** or **remove its row** based on the simple meaning you give your sample dataset.

Do not try to write the whole cleaning process at once. After each cleaning step, print the DataFrame and check whether the change you expected actually happened.